Virgo: On-Demand Protection in 5G Networks Through a Reproducible Malware IoT Testbed

Joachim Schmidt, Leonardo Padial, Anna Padée, Eryk Schiller
Institute of Industrial and IT Engineering (InTECH)
Geneva School of Engineering, Architecture and Landscape (HEPIA)
University of Applied Sciences and Arts of Western Switzerland (HESSO)
email: {name.surname}@hes-so.ch

***Abstract*—We present Virgo, a reproducible B5G security testbed built on srsRAN and Open5GS, with support for integration with the O-RAN Software Community RIC. The present study focuses on L3 traffic analysis and performs offline detection, with on-demand, near-real-time protection left as the target of future work. We deploy a Mirai-derived, fully virtualised malware on IoT-class ARM devices within the testbed and generate SYN-flood attacks over the 5G data plane under controlled conditions. We first show that CICFlowMeter, while effective at capturing the large volume of flows produced by randomised destination-IP/port DDoS campaigns, incurs prohibitive memory cost as the number of concurrent connections grows, a practical limitation for inline deployment. We then evaluate an XGBoost classifier trained exclusively on widely used third-party traces (CIC-DDoS2019) on real Mirai-derived SYN-flood traffic captured in our testbed: the model reliably detects the traffic as malicious (attack-F1 $\approx$ 0.998) but misattributes almost all of it to the LDAP class, since Mirai is absent from the training data, exposing an attribution gap that the literature's prevailing public-dataset evaluation methodology does not reveal. By retraining the same model on locally generated traces, we obtain near-perfect binary attack/benign separation, underscoring the necessity of environment-specific training data. Virgo provides an extensible, fully containerised platform for developing and validating Machine Learning (ML)-driven intrusion-detection strategies under realistic 5G conditions.**

***Index Terms*—5G security, IoT botnets, Mirai, DDoS detection, machine learning, reproducible testbed**

I. Introduction

In this paper, we present Virgo, a fully virtualized and reproducible testbed for studying security threats and Machine-Learning (ML)-based protection mechanisms in next-generation mobile networks. The platform is built on a 5G architecture but is designed with a 6G vision: an open, flexible, and extensible infrastructure capable of dynamically integrating new services and artificial-intelligence models for the detection and mitigation of threats such as Distributed Denial-of-Service (DDoS) attacks. One of Virgo's long-term objectives is to contribute to the definition of standardized interfaces that facilitate the deployment of such services in virtualized and distributed environments, interfaces that remain largely unspecified and whose design is a central focus of this work.

A defining characteristic of the testbed is that every functional component, i.e., the mobile network, the traffic sources, the attack workloads, the traffic-analysis pipeline, and the ML inference engine, executes in software as either a Docker container or a Virtual Machine (VM). The core network is deployed using Open5GS[1], to which a 3GPP-compliant Next-Generation Base Station (gNB) is connected via srsRAN[2], both running as Cloud-native Network Functions (CNFs) within Docker containers. The architecture is also compatible with an O-RAN Software Community near-real-time RIC[3], enabling access to radio-level (L1/L2) statistics. However, the present work focuses exclusively on Layer-3 traffic analysis, and these radio-level statistics are therefore not considered. User Equipment (UE) consists of 5G-compatible Samsung S23 smartphones[4] that provide cellular connectivity to Raspberry Pi 5[5] boards acting as Internet-of-Things (IoT) nodes. These nodes host both legitimate traffic generators and attack workloads, the latter running inside strongly isolated VMs to confine real Mirai-derived malware to the testbed while preserving realistic traffic characteristics over the air interface.

For network-layer threat evaluation, virtualized Mirai bots execute SYN-flood campaigns whose traffic traverses the 5G data plane end-to-end. Captured traffic is processed by CICFlowMeter [1], a widely adopted NetFlow-level feature extractor that computes per-flow statistics suitable for ML-based classification. We show, however, that CICFlowMeter incurs prohibitive memory consumption when confronted with the large number of concurrent micro-flows produced by randomized destination-IP and destination-port DDoS campaigns, a practical bottleneck that is seldom discussed in the literature. Detection is performed by an eXtreme Gradient Boosting (XGBoost) classifier deployed within the containerized pipeline. We first demonstrate that a model trained exclusively on commonly used third-party datasets, when applied to production-representative Mirai SYN-flood traffic captured in Virgo, detects the traffic as malicious but misattributes almost all of it to an unrelated attack class, an attribution failure that is expected given the absence of Mirai from the training data, yet one that prevailing public-dataset evaluation methodologies do not reveal. By retraining the same model on locally generated traces, we obtain near-perfect binary attack/benign separation, un-

[1]https://open5gs.org/
[2]https://www.srsran.com/
[3]https://github.com/srsran/oran-sc-ric
[4]https://www.samsung.com
[5]https://www.raspberrypi.com/products/raspberry-pi-5/

derscoring the necessity of environment-specific training data. Owing to its fully software-defined, modular design, Virgo can accommodate additional detection and mitigation techniques, supporting systematic evaluation of evolving threats toward future 6G mobile systems.

II. Related Work

This section reviews prior work relevant to the Virgo platform. Because Virgo targets cross-layer experimentation, we first briefly situate it among software-defined 5G testbeds that expose lower-layer (L1/L2) radio measurements, and then concentrate on the network-layer (L3) IoT-botnet landscape that is the focus of this paper. Table I compares representative IoT testbeds and traffic generators with Virgo, highlighting its reproducibility, real-device 5G connectivity, and dual capability to generate both legitimate and malicious IoT traffic.

A. Radio-Access Observability

Open-source 5G stacks combining a core network such as Open5GS with a Radio Access Network (RAN) implementation such as srsRAN or OpenAirInterface (OAI)[6], driven by SDR hardware (e.g., USRPs[7]), provide full access to the L1 (PHY) and L2 Medium Access Control (MAC) layers. Virgo's srsRAN-based architecture provides a path toward L1/L2 observability and is compatible with an O-RAN Software Community RIC. These radio-level capabilities are not exercised in the present study, which focuses exclusively on network-layer threats.

B. IoT Botnet Landscape and Testbeds

Mirai and its descendants remain the dominant IoT-botnet threat. Gelgi et al. [2] survey IoT botnets and DDoS attacks, identifying Mirai as central and classifying common vulnerabilities (default credentials, firmware flaws, open services) and mitigations (device hardening, anomaly detection, network filtering). Famera et al. [3] analyse four Mirai descendants, Satori, Mukashi, Moobot, and Sonic, showing how each augments Mirai's architecture with new exploits, propagation strategies, and evasion techniques. The continued prevalence of these variants motivates reproducible testbeds for developing and evaluating mitigation methods.

Existing IoT-botnet testbeds differ chiefly in how faithfully they represent real devices and traffic. Fully virtualized environments emphasise scalability and reproducibility: Sáez-de-Cámara et al. [4] build a GNS3-based laboratory that maps physical topologies onto emulated network devices and VM-based hosts, while Koroniotis et al. [5] (Bot-IoT) and Moustafa [6] (TON_IoT) generate labelled datasets from largely software-emulated devices, the latter adding a layered edge-fog-cloud architecture and multi-source telemetry. Application-layer generators such as IoT-Flock [7] and Node-RED-based setups [8], together with the Message Queuing Telemetry Transport (MQTT)-focused datasets MQTT-IoT-IDS2020 [9] and MQTTset [10], produce reproducible protocol-level traffic but do not capture device firmware or wireless characteristics. Server-side tools (e.g., T-Rex, Ostinato, Genesids) complement device-side generators for high-rate replay and injection [11].

At the other extreme, hardware-based testbeds capture authentic malicious behaviour: Meidan et al. [12] (N-BaIoT) infect nine commercial IoT devices with Mirai and BashLite and detect anomalies via deep autoencoders, and MedBIoT [13] combines real and emulated devices to capture traffic from live Mirai, BashLite, and Torii samples. Edge-IIoTset [14] provides a large-scale realistic IoT/IIoT dataset supporting centralized and federated learning.

Comparatively few testbeds incorporate a real cellular data plane. 5GReplay [15] operates on 5G traffic through rule-based mutation and replay to expose weaknesses in core components, and Carrega et al. [16] present a modular B5G/6G security testbed with automated orchestration, programmable data-plane control, and AI/ML evaluation. Virgo differs from all of the above by executing *real* Mirai-derived malware inside strongly isolated VMs on ARM-based Raspberry Pi 5 devices, whose traffic traverses an end-to-end software-defined 5G data plane and is captured at a centralized point for ML-based analysis, combining device-level authenticity, cellular realism, and full reproducibility.

TABLE I: Comparison of IoT/IIoT testbeds and traffic generators with Virgo.

Platform	Device	5G	Reproducible
[4]	Virtual	No	Yes
[12]	Physical	No	Partially
[5]	Virtual	No	Yes
[6]	Mostly Virtual	No	Yes
[9]	Virtual	No	Yes
[8]	Virtual	No	Yes
[7]	Virtual	No	Yes
[10]	Mostly Virtual	No	Yes
[13]	Mostly Virtual	No	Partially
[14]	Mixed	No	Yes
[15]	N/A	Yes	Yes
[16]	Real	Yes	Yes
Virgo	Real	Yes	Yes

III. Architecture

A. Integrated Cellular–IoT Testbed

Virgo is a complete end-to-end 5G testbed (cf. Fig. 1) comprising a 5G Core Network (5GCN), a 5G RAN, and User Equipments (UEs) that carry both legitimate and malicious IoT traffic. Every functional block, i.e., the 5GCN, the RAN, the traffic sources and sinks, the malware, the capture stage, and the ML pipeline, executes in software as a Docker container or a VM, making the platform fully reproducible. The source code and deployment resources for the Virgo testbed are publicly available on GitHub.[8]

[6]https://openairinterface.org/

[7]https://www.ettus.com/products/

[8]https://github.com/orgs/Hepia/teams/virgo/

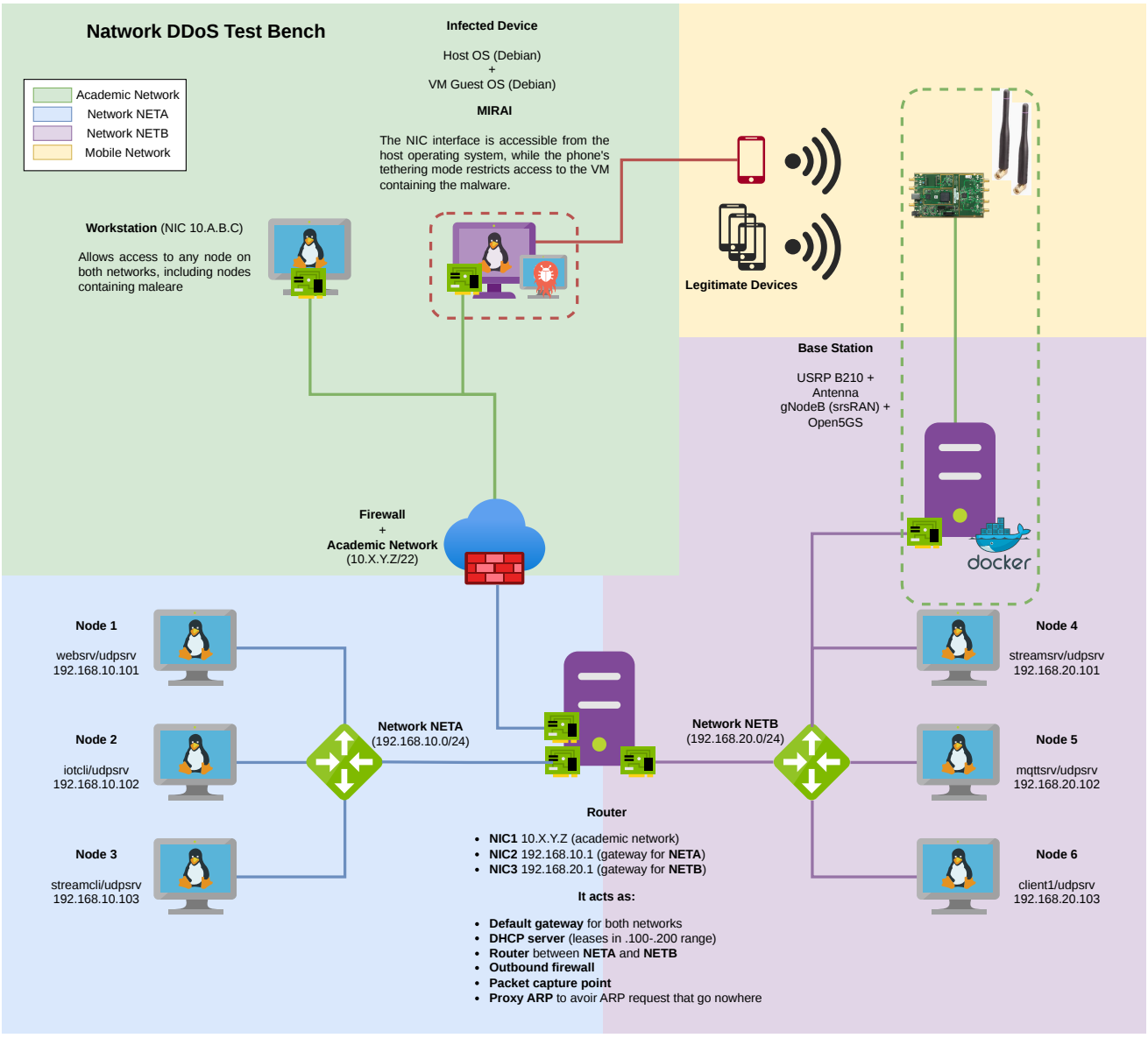


Fig. 1: End-to-end Virgo testbed architecture, comprising the 5G infrastructure, legitimate smartphones and IoT devices, and isolated Mirai-infected nodes connected through the experimental mobile network.

The 5GCN is deployed with containerised Open5GS and includes the Network Slice Selection Function (NSSF), Network Exposure Function (NEF), Network Repository Function (NRF), Policy Control Function (PCF), Unified Data Management (UDM), Application Function (AF), Authentication Server Function (AUSF), Access and Mobility Management Function (AMF), User Plane Function (UPF), and Service Communication Proxy (SCP). The RAN is provided by a containerised srsRAN_project gNB[9], with radio transmission handled by a USRP B210 software-defined radio serving as the gNB transceiver. The srsRAN-based architecture is also compatible with an O-RAN Software Community near-real-time RIC[10] through the E2 interface. This capability is not exercised in the present study, which focuses exclusively on L3 threats.

Commercial Samsung S23[11] smartphones, fitted with Osmocom-programmable SJA5-9FV SIM cards[12] configured for the local Open5GS network, act as UEs and provide IP connectivity to attached Raspberry Pi (RPI) 5 devices via USB tethering. This design lets us inject arbitrary IoT traffic,

[9]https://www.srsran.com/

[10]https://github.com/srsran/oran-sc-ric

[11]https://www.samsung.com

[12]https://shop.sysmocom.de

whether legitimate or malicious, into the test network directly over the air interface. The gNB and UEs operate in the N77 band with a 10 MHz channel bandwidth; it is strongly recommended that the setup be operated inside a Faraday cage. As shown in Fig. 1, six IoT nodes (*Node 1–Node 6*) host legitimate client and server services, while the base station simultaneously serves these legitimate devices and a Mirai botnet that can be activated on demand.

B. Traffic Generation and Malware Containment

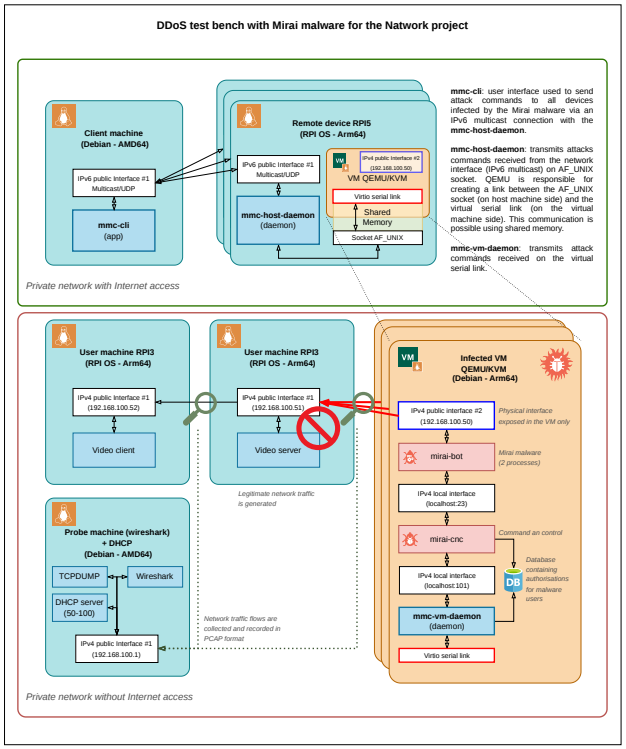


Fig. 2: Mirai isolation, management, and control.

Legitimate traffic. Legitimate devices can be deployed either as standard devices or as lightweight Docker containers [13] connected to the test interface exposed by the tethered smartphone, sharing the same network stack as ordinary IoT endpoints. The testbed supports representative application patterns: MQTT via Mosquitto[14], UDP through Python socket programming, HTTP with nginx[15]/curl[16], and video streaming with mediamtx[17]/ffplay[18]. Containers are assigned explicit source or sink roles, and multiple containers per node yield reproducible yet non-trivial traffic dynamics (e.g., a source may target a sink, or a destination where no sink is deployed, enabling stress scenarios). In addition, the setup includes legitimate traffic generated by a human user interacting with a smartphone connected to a base station within one of the experimental mobile networks, providing realistic user-driven network activity alongside the emulated IoT traffic.

Malicious traffic. A key feature of Virgo is that it runs *real* Mirai-derived malware rather than synthetic attack traffic. Each RPI 5 hosts a dedicated QEMU/KVM virtual machine[19] that confines the malware. The host system of the "infected" machine is equipped with two interfaces: one visible to the host operating system for administrative management over the regular (e.g., academic) network, and one dedicated exclusively to the VM and connected to the isolated test network. The VM is connected to the experimental mobile network through a tethered smartphone, ensuring that Mirai traffic traverses the same cellular infrastructure as legitimate devices while remaining isolated from external networks. No NAT or bridging is enabled on the latter, ensuring the malware remains fully contained and cannot propagate beyond the testbed. The compute capacity of the RPI 5 makes virtualization-based isolation of live malware practical, allowing a controlled Mirai botnet to generate realistic traffic within the enclosure.

Malware Management and Control (MMC). The malicious VMs are orchestrated by a dedicated MMC framework (cf. Fig. 2), composed of the newly implemented *mmc-vm-daemon*, *mmc-host-daemon*, and *mmc-cli* applications. The QEMU/KVM layer exposes an AF_UNIX socket[20] on the RPI 5 host side and a virtio serial console on the RPI 5 guest side; *mmc-vm-daemon* relays instructions to the Mirai command-and-control agent inside the VM, *mmc-host-daemon* forwards operator commands from the management network to the VM through the AF_UNIX socket, and *mmc-cli* discovers and controls hosts over a multicast management channel. This separates the control plane from the malicious data plane, so that attacks can be started, stopped, and reproduced under strict containment.

C. Observation and Processing Pipeline

1) Observation point: All test traffic is routed through a central node that simultaneously acts as router, DHCP server, and outbound firewall (cf. Fig. 1). Traffic can be captured at this centralized packet-capture point or, equally, at any stage of the 5G network, from the RAN through the user plane between the gNB and the UPF, using `tcpdump`[21] and `Wireshark`[22]. Centralizing observation at a single point yields authoritative ground-truth traces and simplifies labeling based on IPs; the captured data can then be processed offline for dataset generation or online for real-time detection.

2) Trace processing pipeline: Captured traffic is processed by a fully containerised pipeline (cf. Fig. 3) to guarantee reproducibility and portability. Raw packet captures (PCAP) are converted into bidirectional flow records by CICFlowMeter V3[23] [1], which enriches each flow with statistical features

[13] https://www.docker.com/
[14] https://mosquitto.org/
[15] https://nginx.org/
[16] https://curl.se/
[17] https://github.com/bluenviron/mediamtx
[18] https://ffmpeg.org/ffplay.html
[19] https://www.qemu.org
[20] https://man7.org/linux/man-pages/man7/unix.7.html
[21] https://www.tcpdump.org/
[22] https://www.wireshark.org/
[23] https://github.com/ahlashkari/CICFlowMeter

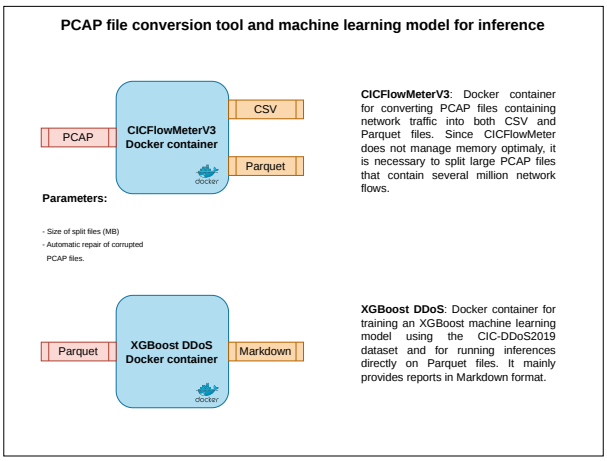


Fig. 3: PCAP-to-flow conversion, XGBoost training, and inference.

such as flow duration, inter-arrival times, byte and packet counts, and burst metrics. The resulting flows are exported in CSV and Parquet[24] formats, then cleaned, label-encoded, and feature-scaled into vectors of 77 attributes per flow. The schema is harmonized with widely used public datasets to enable direct comparison. A dedicated container runs Python-based XGBoost [17] models; training and inference are currently performed offline, and the platform does not yet employ hardware acceleration.

D. Datasets and Detection Strategy

Virgo is designed both to consume public reference data and to generate its own confined traces. The evaluation therefore follows a two-fold strategy that isolates the effect of the training source. As an external reference, we use the CIC-DDoS2019 dataset[25], comprising 431,371 labelled instances and 77 features spanning legitimate traffic and a diverse set of DDoS attacks (UDP, TCP-SYN, LDAP, MSSQL, NTP, and TFTP floods). As internal data, we collect Mirai traces directly from our testbed, pre-processed with the same pipeline so that training and evaluation share an identical schema.

We then evaluate two configurations. First, an XGBoost model is trained on the third-party CIC-DDoS2019 data and applied to our locally captured Mirai traffic, probing whether public-dataset training transfers to real Mirai-derived attacks generated within the testbed. Second, the same model is retrained on our own testbed traces and again evaluated on Mirai traffic, measuring in-domain detection. This design lets us quantify the domain-shift gap while keeping the testbed itself, i.e., a reproducible, confined infrastructure for generating realistic and varied datasets, as the central contribution. Owing to its modularity, Virgo can host detection models beyond XGBoost and, in line with its 6G vision, deploy them on demand as interface standardization matures.

[24]https://parquet.apache.org/

[25]https://www.unb.ca/cic/datasets/ddos-2019.html

IV. Results

This section evaluates the network-layer DDoS pipeline of Virgo, whose flow extraction (CICFlowMeter), Parquet storage, and XGBoost training and inference each run in dedicated Docker containers. We first expose a practical limitation of flow extraction under Mirai traffic, then quantify the domain-shift gap between a model trained on the public CIC-DDoS2019 dataset and one trained on local traces, and finally argue that even the near-perfect in-domain scores rest on a shallow discriminative signal. In our experiments, we generated around 45 GB of traffic in Virgo, and reused 28 GB of traffic from the CIC-DDoS2019 dataset in PCAP files.

A. Flow-Extraction Cost under Mirai Traffic

Mirai SYN-flood campaigns are dominated by extremely short-lived flows: a typical attack flow is a single forward SYN and, at most, one returning segment. When the campaign targets randomised destination addresses, most SYNs reach non-existent hosts, so no SYN/ACK is returned and the flow never completes. CICFlowMeter, which holds per-flow state until a flow terminates or times out, must therefore keep a very large number of open, single-packet records simultaneously, producing a state explosion specific to randomised, connection-incomplete attacks. As shown in Fig. 4, for two comparably sized captures, a 39.2 MB legitimate trace (HTTP/MQTT/RTSP) and a 57.8 MB Mirai SYN trace, peak memory rises from 227 ± 12 MB to 1715 ± 51 MB, a sevenfold increase despite only modest input growth. Because peak memory scales with the number of concurrent flows rather than byte volume, extraction over long attack captures can exhaust memory outright; we therefore split captures into chunks of at most 64 MB before extraction. This bottleneck, seldom reported in the literature, directly affects any inline deployment of CICFlowMeter-style extraction against modern IoT-botnet traffic.

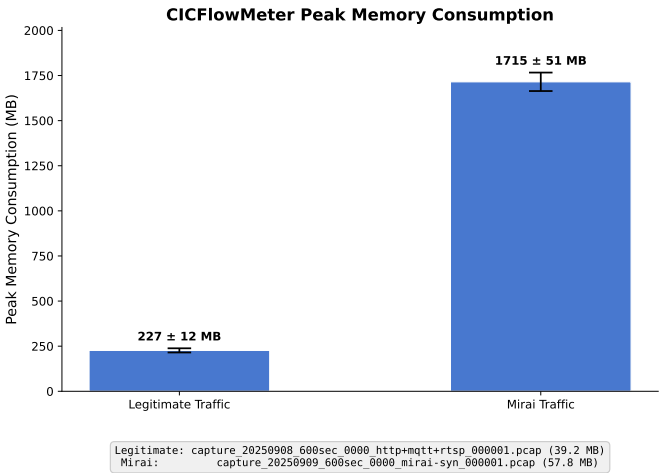


Fig. 4: CICFlowMeter peak memory on legitimate vs. Mirai SYN captures.

B. Detection with Third-Party Data (CIC-DDoS2019)

We train XGBoost (100 estimators, depth six, learning rate 0.1, subsample and column-subsample 0.8) on CIC-DDoS2019 and apply it, without adaptation, to local Mirai traces. As Mirai is not a class in CIC-DDoS2019, the captured SYN-flood flows receive the closest placeholder ground-truth label (`Syn`) for scoring. The confusion matrix (Fig. 5) exhibits a clear domain shift: almost none of the Mirai flows are attributed to their placeholder class, and 158,839 of 159,329 are instead predicted as `LDAP`, while some benign flows are pushed into attack classes. As a multi-class attack-type classifier the model effectively fails, with only 39 of 160,026 samples on the diagonal (accuracy $\approx 0.02\%$) and a macro-F1 of ≈ 0.02. The failure is one of *attribution*, not *detection*: collapsing the matrix to a binary attack/benign decision gives attack precision 0.996 and recall ≈ 1.000 (attack-F1 ≈ 0.998). The model reliably flags the traffic as malicious but consistently mislabels a Mirai SYN flood as an LDAP attack. Since Mirai is absent from the training classes, a correct label is unattainable by construction; the informative finding is therefore not the misattribution itself but the fact that the flows are absorbed by `LDAP` rather than by the structurally closest class, `Syn`, which indicates that the learned decision boundaries do not transfer to a new environment and that public-dataset evaluation overstates operational robustness.

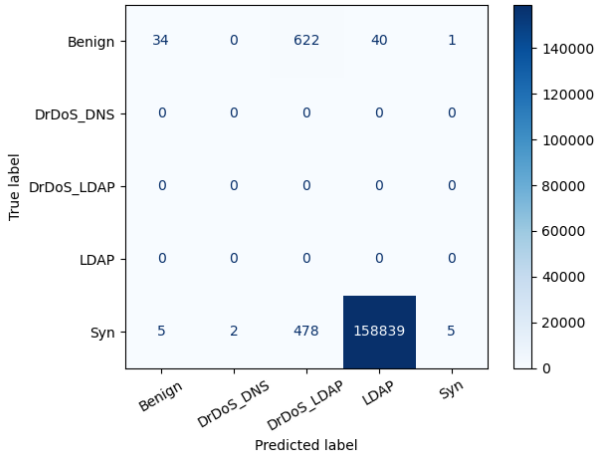


Fig. 5: CIC-DDoS2019-trained model applied to local Mirai traces.

C. Detection with Locally Generated Data

Retraining the identical configuration on Virgo-generated traces, with the same feature schema and pre-processing so that only the training source changes, yields near-perfect binary separation (Fig. 6). Because only benign and Mirai SYN-flood traffic are available locally, this model is a two-class benign/DDoS classifier rather than a multi-class attack-type classifier. Training and test flows are drawn from disjoint captures: the model is trained on a separate set of Virgo traces and evaluated on the same 160,026 held-out flows used in Sect. IV-B, so that no flow appears in both sets. DDoS flows are recovered almost exactly (precision 1.000, recall 0.9999), and benign flows reach precision 0.986 and recall 0.993, limited only by their small support of 697 samples. The overall accuracy is 99.99% with a macro-F1 of 0.995. The two experiments are not directly comparable as attack-type classifiers, since the third-party model solves a multi-class attack-type problem (macro-F1 ≈ 0.02) while the local model solves a binary one; the like-for-like comparison is the binary attack/benign decision, on which the local model improves benign precision and recall from the cross-domain case and removes the systematic misattribution. Matching training and deployment domains is thus the decisive factor.

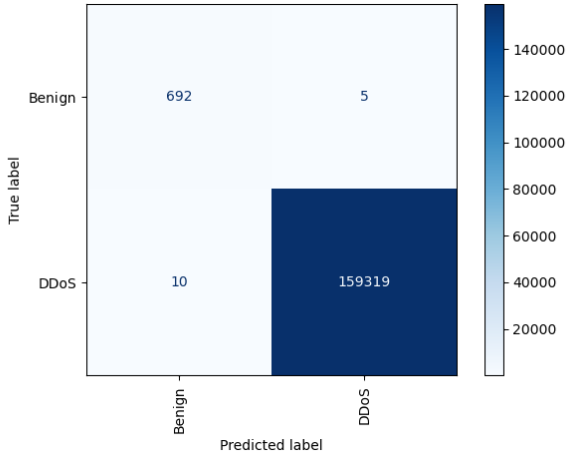


Fig. 6: Model trained and tested on local Virgo data.

D. What the Classifier Actually Detects

The near-perfect in-domain scores should be read with care. Mirai SYN flows are degenerate: a representative flow carries one forward and maybe one backward packet, zero payload bytes, a duration of a few hundred microseconds, and only SYN/ACK/RST flags. Such flows are trivially separable from legitimate HTTP, MQTT, or RTSP traffic, which involves many packets, non-zero payloads, and longer durations. The classifier therefore learns a low-level structural signature ("short, payload-less, SYN-bearing flow") rather than a semantic model of attack behaviour, which explains both the high in-domain score and the fragility of Sect. IV-B: once the attack variant or flow-shaping tool changes, the signature no longer transfers. This motivates self-generated datasets and temporal or hybrid features that capture cross-flow Mirai behaviour, and underlines the value of Virgo for exposing exactly the domain-shift and shallow-signal effects that public-dataset evaluations tend to hide.

V. Summary

This paper presented Virgo, a fully virtualised and reproducible 5G security testbed in which the mobile network, traffic sources, real Mirai-derived malware, and the machine-learning pipeline all run as containers or isolated virtual machines. Its central contribution is exposing the domain-shift gap that dominates network-layer intrusion detection:

an eXtreme Gradient Boosting (XGBoost) model trained on the widely used CIC-DDoS2019 dataset reliably flags Mirai traffic as malicious yet almost entirely mislabels it, whereas retraining on locally generated traces restores near-perfect in-domain detection. We further showed that CICFlowMeter incurs prohibitive memory cost under randomised Mirai floods, and that even high in-domain scores rest on a shallow structural signature, underscoring the need for representative, self-generated datasets. Future work will enable near-real-time inference and extend Virgo toward programmable security services for emerging Sixth-Generation (6G) systems.

ACKNOWLEDGEMENTS

This work was carried out within the framework of the NATWORK project—Net-Zero Self-Adaptive Activation of Distributed Self-Resilient Augmented Services. NATWORK has received funding from the European Union's Horizon Europe research and innovation programme through the Smart Networks and Services Joint Undertaking (SNS JU) under Grant Agreement No. 101139285, with additional support from the Swiss State Secretariat for Education, Research and Innovation (SERI) and UK Research and Innovation (UKRI). The authors gratefully acknowledge this support.

REFERENCES

[1] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of tor traffic using time based features," in *Proceedings of the 3rd International Conference on Information Systems Security and Privacy - Volume 1: ICISSP,*, INSTICC. SciTePress, 2017, pp. 253–262.

[2] M. Gelgi, Y. Guan, S. Arunachala, M. Samba Siva Rao, and N. Dragoni, "Systematic literature review of iot botnet ddos attacks and evaluation of detection techniques," *Sensors*, vol. 24, no. 11, 2024. [Online]. Available: https://www.mdpi.com/1424-8220/24/11/3571

[3] A. Famera, B. Hilger, S. Bhunia, and P. Heil, "Analyzing the mirai iot botnet and its recent variants: Satori, mukashi, moobot, and sonic," *arXiv preprint arXiv:2508.01909*, 2025.

[4] X. Sáez-de Cámara, J. L. Flores, C. Arellano, A. Urbieta, and U. Zurutuza, "Gotham testbed: A reproducible iot testbed for security experiments and dataset generation," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 1, pp. 186–203, 2024.

[5] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset," *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X18327687

[6] N. Moustafa, "A new distributed architecture for evaluating ai-based security systems at the edge: Network ton_iot datasets," *Sustainable Cities and Society*, vol. 72, p. 102994, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2210670721002808

[7] S. Ghazanfar, F. Hussain, A. U. Rehman, U. U. Fayyaz, F. Shahzad, and G. A. Shah, "Iot-flock: An open-source framework for iot traffic generation," in *2020 International Conference on Emerging Trends in Smart Technologies (ICETST)*, 2020, pp. 1–6.

[8] M. Lekić and G. Gardašević, "Iot sensor integration to node-red platform," in *2018 17th International Symposium INFOTEH-JAHORINA (INFOTEH)*, 2018, pp. 1–5.

[9] H. Hindy, E. Bayne, M. Bures, R. Atkinson, C. Tachtatzis, and X. Bellekens, "Machine learning based iot intrusion detection system: An mqtt case study (mqtt-iot-ids2020 dataset)," in *Selected Papers from the 12th International Networking Conference*, B. Ghita and S. Shiaeles, Eds. Cham: Springer International Publishing, 2021, pp. 73–84.

[10] I. Vaccari, G. Chiola, M. Aiello, M. Mongelli, and E. Cambiaso, "Mqttset, a new dataset for machine learning techniques on mqtt," *Sensors*, vol. 20, no. 22, 2020. [Online]. Available: https://www.mdpi.com/1424-8220/20/22/6578

[11] M. Swann, J. Rose, G. Bendiab, S. Shiaeles, and N. Savage, "Tools for network traffic generation–a quantitative comparison," *arXiv preprint arXiv:2109.02760*, 2021.

[12] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, A. Shabtai, D. Breitenbacher, and Y. Elovici, "N-baiot—network-based detection of iot botnet attacks using deep autoencoders," *IEEE Pervasive Computing*, vol. 17, no. 3, pp. 12–22, 2018.

[13] A. Guerra-Manzanares, J. Medina-Galindo, H. Bahsi, and S. Nõmm, "Medbiot: Generation of an iot botnet dataset in a medium-sized iot network." in *ICISSP*, 2020, pp. 207–218.

[14] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-iiotset: A new comprehensive realistic cyber security dataset of iot and iiot applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40 281–40 306, 2022.

[15] Z. Salazar, F. Zaïdi, H.-N. Nguyen, W. Mallouli, A. R. Cavalli, and E. M. D. Oca, "A network traffic mutation based ontology, and its application to 5g networks," *IEEE Access*, vol. 11, pp. 43 925 – 43 944, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10105914

[16] A. Carrega, F. Davoli, and R. Rabbani, "B5g/6g cyber security testbed," in *Way to 6G Workshop (WT6G) on 33rd International Symposium on the Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*. Paris, France: IEEE, October 2025, creative Commons Attribution 4.0 (CC-BY 4.0). [Online]. Available: https://zenodo.org/record/17393071

[17] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 785–794. [Online]. Available: https://doi.org/10.1145/2939672.2939785